# 02 - decision trees, RF, DNNs

| | |
|---|---|
| Tags | |
| Course | Machine Lerning |
| Confidence | Not Confident |
| Last Edited | 7 October 2026 11:08 |

# Dispensa di Studio: Machine Learning and Security

## Modulo III: Decision Trees, Random Forests e Deep Neural Networks (DNNs)

### MODULO III — PARTE 1: Alberi di Decisione (Decision Trees)

#### 1. Concetti Chiave e Definizioni Formali

- **Decision Tree (Albero di Decisione):** Modello di apprendimento supervisionato rappresentato come una struttura gerarchica ad albero invertito. Consiste in una collezione organizzata di test logici o condizioni (detti *splits* o *tests*) in cui ogni nodo interno esegue una verifica su una o più feature dell'input e ogni nodo foglia restituisce una predizione.
- **Anatomia dell'Albero (Nodi e Architettura):**
  - **Root Node (Nodo Radice):** Il nodo posizionato al vertice dell'albero che accoglie l'intero dataset iniziale.
  - **Non-Leaf / Internal Node (Nodo Interno):** Nodo intermedio che contiene una condizione/test valutata su uno o più attributi.
  - **Leaf Node (Nodo Foglia / Terminale):** Nodo privo di figli che rappresenta l'output del modello (un'etichetta di classe discreta per la classificazione o un valore reale per la regressione).
- **Condizioni Axis-Aligned vs. Oblique:**
  - **Axis-Aligned (Ortogonali agli assi):** Condizioni che valutano una singola feature per volta (es. $x_i \geq \theta$). Generano confini di decisione costituiti da iperpiani parallelamente allineati agli assi dello spazio delle feature.
  - **Oblique (Inclinati):** Condizioni che combinano linearmente molteplici feature (es. $\sum w_i x_i \geq \theta$), consentendo di tracciare confini di separazione iperplanari inclinati.
- **Alberi Binari vs. Non-Binari:**
  - **Binary Decision Trees:** Ciascun nodo interno si suddivide in esattamente due rami figli (corrispondenti alle risposte logiche Yes/No o True/False). Rappresentano la struttura fondante dei modelli ensemble come le Random Forests.
  - **Non-Binary Decision Trees:** Ciascun nodo può ramificarsi in un numero arbitrario e variabile di sotto-nodi (es. per feature categoriche con molteplici valori discreti). Una ramificazione

eccessiva aumenta tuttavia il rischio di overfitting.

- **Entropia di Shannon ($H(D)$):** Misura dell'impurità, del disordine e del grado di incertezza presente in una collezione di dati $D$ rispetto a $C$ classi target:

$$H(D) = -\sum_{i=1}^{C} y_i \log_2(y_i)$$

dove $y_i$ indica la frazione di campioni appartenenti alla classe $i$-esima nel nodo.

- **Valore Massimo:** Si verifica quando le classi sono perfettamente miscelate ed equiprobabili (es. $H = 1.0$ in un problema binario con distribuzione $50\%/50\%$).
- **Valore Minimo (Nodo Puro):** Si azzera ($H = 0.0$) quando tutti i campioni appartengono ad un'unica classe.
- **Information Gain ($G(D, f)$ - Guadagno di Informazione):** La riduzione attesa dell'entropia ottenuta partizionando l'insieme di campioni $D$ lungo la feature $f$:

$$G(D, f) = \text{Entropy}(D) - \sum_{v \in \text{values}(f)} \frac{|D_v|}{|D|} \text{Entropy}(D_v)$$

dove $D_v$ rappresenta il sottoinsieme di $D$ per cui la feature $f$ assume il valore $v$.

---

**2. Trattazione Teorica Estesa**

**A. Il Problema della Costruzione Ottimale e l'Approccio Greedy**

La ricerca dell'albero di decisione globale di dimensione minima che minimizza l'errore di classificazione è dimostrata essere un problema di complessità computazionale **NP-hard**. Di conseguenza, l'addestramento pratico si affida a strategie euristiche di tipo *Greedy Divide-and-Conquer* (utilizzate negli algoritmi classici ID3, C4.5 e CART).

**B. Algoritmo di Crescita Ricorsiva**

1. **Inizializzazione:** Il processo parte dal nodo radice contenente l'intero insieme di addestramento $D$.
2. **Valutazione e Selezione dello Split:** Ad ogni nodo interno, l'algoritmo esamina le feature disponibili e valuta le possibili condizioni di suddivisione. Viene selezionata la condizione che massimizza la metrica di purezza prescelta (es. l'Information Gain $G(D, f)$).
3. **Gestione dei Tipi di Feature:**
   - **Feature Discrete/Categoriche:** L'Information Gain viene calcolato valutando i sottoinsiemi generati dai singoli valori discreti.
   - **Feature Continue:** I valori continui della feature vengono ordinati in modo crescente; l'algoritmo valuta i punti medi tra valori adiacenti come possibili soglie $x_i \geq \theta$, identificando la soglia ottima che garantisce il massimo valore di $G(D, f)$.
4. **Condizioni di Arresto (Stopping Criteria):** La ricorsione si interrompe se:
   - Il sottoinsieme associato al nuovo nodo raggiunge una purezza elevata o assoluta ($H \approx 0$).

- Non vi sono ulteriori feature disponibili per effettuare ulteriori partizioni.
- Si raggiunge un limite prefissato sulla profondità massima dell'albero (`max_depth`) o sul numero minimo di campioni per foglia (`min_samples_leaf`), prevenendo l'overfitting.

---

### 3. Focus su Schemi, Diagrammi e Immagini delle Slide

#### A. Decostruzione dell'Albero Binario vs Non-Binario

```
      [Albero Binario]                          [Albero Non-Binario]

    num_legs ≥ 3 ?                                  num_legs
     /          \                                 /    |    \
  (Yes)         (No)                             8     4     2
   /              \                             /      |      \
num_eyes ≥ 3 ?   [penguin]                  [spider] [dog] [penguin]
 /         \
(Yes)      (No)
 /           \
[spider]     [dog]
```

- **Decostruzione Analitica:**
  - Nell'**albero binario** (a sinistra), ogni decisione è una proposizione booleana. Il nodo radice testa `num_legs ≥ 3`. Se falso (*No*), si isola immediatamente la classe *penguin*. Se vero (*Yes*), un secondo nodo valuta `num_eyes ≥ 3`, separando *spider* (*Yes*) da *dog* (*No*).
  - Nell'**albero non-binario** (a destra), il nodo `num_legs` genera tre rami distinti corrispondenti ai valori discreto-numerici 8, 4 e 2. Sebbene riduca la profondità dell'albero, l'approccio non-binario frammenta rapidamente il dataset ad ogni split, causando un rapido svuotamento dei nodi figli e aumentando fortemente il rischio di overfitting.

#### B. Decostruzione del Diagramma dell'Entropia e Disordine

- **Analisi della Figura:**
  - **High Entropy:** I campioni blu e arancioni sono mescolati in modo completamente omogeneo e caotico (disordine massimo, $H \approx 1.0$).
  - **Low Entropy:** I punti arancioni costituiscono la quasi totalità della popolazione, con isolati punti blu residui ($H \approx 0.3$).
  - **No Entropy (Pure):** Il riquadro contiene esclusivamente triangoli arancioni ($H = 0.0$). Il nodo è perfettamente puro.
- **Significato Geometrico:** L'addestramento dell'albero è una trasformazione sequenziale che suddivide lo spazio delle feature per passare da regioni ad elevata entropia a regioni trascurabili o nulle (foglie pure).

---

### 4. Esempi Applicativi e Note di Approfondimento

- **Applicazione in Cybersecurity (Rule-based IDS):** Un albero di decisione addestrato su log di traffico di rete può apprendere regole interpretabili per il rilevamento di scansioni di porte (*Port Scan*). Ad esempio:

$$\text{if } (\text{packet_length} \leq 40 \text{ bytes}) \land (\text{TCP_flags_SYN} = 1) \implies \text{Classificata come "Port Scan"}$$

La struttura gerarchica dell'albero permette agli analisti di sicurezza di convertire direttamente i nodi in regole esplicite per firewall o sistemi IDS.

---

## MODULO III — PARTE 2: Decision Forests e Random Forest

### 1. Concetti Chiave e Definizioni Formali

- **Ensemble Methods (Metodi Ensemble):** Paradigma di machine learning che combina molteplici classificatori individuali (*base learners* o *weak learners*) in un unico modello predittivo finale per migliorare la capacità di generalizzazione e ridurre il tasso d'errore.
- **Random Forest (RF):** Algoritmo ensemble appartenente alla famiglia del **Bagging (Bootstrap Aggregating)** che edifica una foresta di alberi di decisione binari non potati (*unpruned decision trees*) addestrati in modo indipendente.
- **Bootstrapping (Ricampionamento Bootstrap):** Processo di campionamento stocastico con reinserimento (*sampling with replacement*) condotto sul dataset di addestramento originale. Estrae $k$ sottoinsiemi di dimensione $N$ a partire da un dataset iniziale di $N$ campioni.
- **Metodo del Vettore Casuale (Random Vector Method / Feature Subspacing):** Ad ogni nodo di ciascun albero, la scelta del miglior criterio di splitting non avviene sull'intero insieme delle $p$ feature, ma su un sottoinsieme estratto casualmente di $m$ feature (con $m \ll p$, tipicamente $m = \sqrt{p}$).
- **Out-Of-Bag (OOB) Error:** Stima dell'errore di generalizzazione calcolata valutando le predizioni di ciascun albero esclusivamente sui campioni del dataset d'addestramento che non sono stati inclusi nel suo rispettivo campione bootstrap (circa il $36.8\%$ dei dati d'origine).

---

### 2. Trattazione Teorica Estesa

#### A. Fondamenti Matematici dell'Ensemble e Riduzione della Varianza

Se si considerano $B$ classificatori indipendenti e identicamente distribuiti (*i.i.d.*), ciascuno con varianza $\sigma^2$, la varianza della media delle loro predizioni è pari a:

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{B}$$

Tuttavia, gli alberi di decisione addestrati sul medesimo dataset tendono ad essere correlati. Se la correlazione media tra le predizioni di due alberi è $\rho > 0$, la varianza complessiva dell'ensemble assume la forma:

$$\text{Var}(\text{RF}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

```
                        RIDUZIONE DELLA VARIANZA IN RF
Varianza Singolo Albero: σ²
Varianza Forest:         Var(RF) = ρσ² + ((1 - ρ) / B) σ²

Obiettivo Random Forest:
```

```
1. Aumentare B (Numero di alberi nella foresta)  ⟶  ((1 - ρ) / B) σ²  ⟶  0
2. Ridurre ρ (Correlazione tra alberi via Subspacing) ⟶  ρσ² Minimizzato
```

Il successo della Random Forest deriva dalla capacità di ridurre al minimo il fattore di correlazione $\rho$ mediante due meccanismi di casualità:

1. **Bootstrapping sui dati:** Ciascun albero apprende su una combinazione leggermente diversa di campioni.
2. **Random Feature Subspacing:** Ciascun nodo valuta uno spazio di feature differente, impedendo ad attributi fortemente dominanti di guidare la radice di tutti gli alberi e rendendo i modelli altamente diversificati.

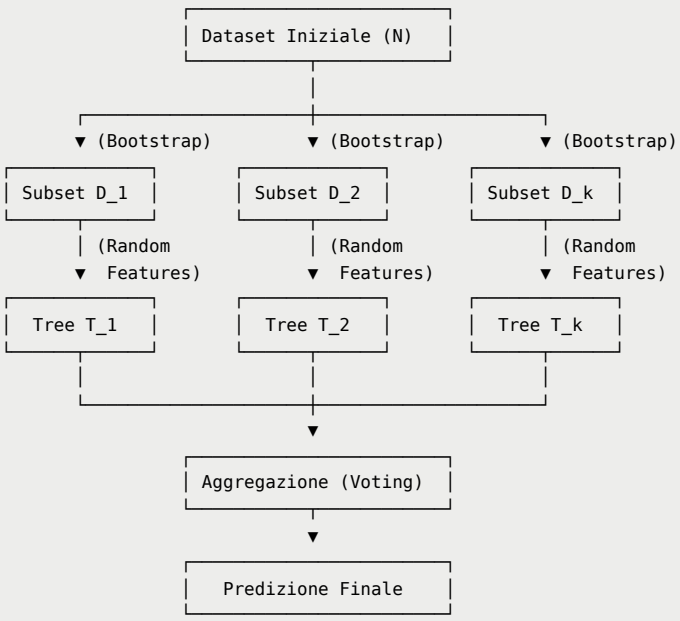
**B. Fasi dell'Algoritmo Random Forest**

1. Estrarre $k$ sottoinsiemi di addestramento mediante ricampionamento bootstrap con reinserimento.
2. Per ciascun sottoinsieme, addestrare un albero di decisione senza potatura (*unpruned tree*): ad ogni nodo, selezionare casualmente $m$ feature e identificare la condizione ottima basata sull'Information Gain.
3. Calcolare la stima dell'errore OOB mediante le istanze escluse dal bootstrap.
4. Aggregare le predizioni di tutti i $k$ alberi tramite **Majority Voting** (votazione a maggioranza) per la classificazione o tramite media aritmetica per la regressione.

---

**3. Focus su Schemi e Logica Operativa**

**Flusso Operativo della Random Forest**




- **Decostruzione dei Vantaggi:**

- **Decorrelazione:** L'indipendenza strutturale degli alberi riduce la probabilità di overfitting all'aumentare di $k$.
- **Basso Costo Computazionale:** L'addestramento dei singoli alberi su sottoinsiemi ridotti di feature è altamente parallelizzabile e notevolmente più veloce rispetto all'addestramento di reti neurali profonde.

---

## MODULO III — PARTE 3: Deep Neural Networks (DNNs) e Funzioni d'Attivazione

### 1. Concetti Chiave e Definizioni Formali

- **Deep Neural Network (DNN):** Termine ombrello che identifica le architetture di apprendimento profondo composte da una sequenza stratificata di trasformazioni funzionali. Include Deep Feedforward Networks (MLP), Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), GANs e Transformers.
- **Deep Feedforward Network / Multi-Layer Perceptron (MLP):** Modello aciclico rappresentabile come un grafo diretto orientato privo di cicli (DAG) che definisce una mappatura $\hat{y} = f(x; \theta)$. È una composizione di $L$ funzioni stratificate:

$$f(x) = f^{(L)}(f^{(L-1)}(\dots f^{(1)}(x) \dots))$$

La lunghezza della catena di composizioni definisce la profondità (*depth*) della rete.

- **Approssimazione Funzionale e Hidden Layers:** La rete cerca di approssimare una funzione sconosciuta $y = f^*(x)$. Durante l'addestramento supervisionato sui dati $(x_i, y_i)$, gli hidden layers modificano internamente le proprie rappresentazioni per consentire all'ultimo strato di produrre un valore il più vicino possibile al target reale.
- **Modello Lineare su Rappresentazione Non Lineare:**

$$\hat{y} = f(x; \theta, w) = \phi(x; \theta)^T w$$

I livelli nascosti apprendono la funzione $\phi(x; \theta)$, che trasforma l'input in uno spazio intermedio di feature dove i dati risultano linearmente separabili dal vettore di pesi finale $w$.

---

### 2. Trattazione Teorica Estesa: Tassonomia delle Funzioni d'Attivazione ($g(z)$)

La funzione d'attivazione $g(z)$ viene applicata al termine pre-attivazione $z = w^T x + b$ per introdurre la non-linearità fondamentale nel sistema.

| Funzione d'Attivazione | Formulazione Matematica | Caratteristiche & Ambiti d'Uso |
|---|---|---|
| **Linear** | $g(z) = \lambda z + b$ | Inadatta agli hidden layer: rende la composizione puramente lineare. |
| **Logistic Sigmoid** | $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$ | Mappa l'input in $(0, 1)$. Usata in output per classificazione binaria. |
| **ReLU (Rectified Linear)** | $g(z) = \max(0, z)$ | Non-lineare a tratti; derivata unitaria per $z > 0$. Standard negli hidden layer. |
| **Softmax** | $g(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ | Multivariata; normalizza l'output in una distribuzione di probabilità ($[0, 1]$ con somma 1). |

**Analisi Dettagliata delle Singole Funzioni**

- **Linear Activation ($g(z) = \lambda z + b$):** Se utilizzata in tutti i livelli, l'intera composizione si riduce a una trasformazione affine globale: $f(x) = W_{\text{tot}}x + b_{\text{tot}}$. Non consente di apprendere confini di decisione complessi o non lineari.
- **Logistic Sigmoid:** Approssima in modo liscio (*smooth*) la funzione a gradino (*step function*). È differenziabile su tutto il dominio reale, proprietà essenziale per la Backpropagation. Tuttavia, per $|z| \gg 0$, la derivata $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ si azzera, saturando il neurone e causando il fenomeno del *vanishing gradient*.
- **ReLU (Rectified Linear Unit):** $g(z) = \max(0, z)$. Rappresenta lo standard negli hidden layer delle reti feedforward. Conserva molte proprietà delle funzioni lineari che facilitano l'ottimizzazione basata sul gradiente, prevenendo la saturazione nella regione positiva $z > 0$.
- **Softmax:**

$$g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Funzione multivariata utilizzata negli strati di uscita per problemi di classificazione multi-classe. Trasforma un vettore di logits non scalati $z$ in un vettore di probabilità positive la cui somma è esattamente $1.0$.

---

**3. Focus su Schemi e Diagrammi delle Slide**

**A. Neurone Biologico vs Perceptron Artificiale**

- **Dendriti $\leftrightarrow$ Ingressi ($x_1, x_2, x_3$):** Canali di ricezione dei segnali in ingresso.
- **Soma / Corpo Cellulare $\leftrightarrow$ Combinazione Lineare ($z = \sum w_i x_i + b$):** Integratore dei segnali ponderati.
- **Efficacia Sinaptica $\leftrightarrow$ Pesi ($w_1, w_2, w_3$):** Parametri regolabili che amplificano o attenuano ciascun segnale.
- **Soglia di Firing $\leftrightarrow$ Funzione d'Attivazione ($g(z)$):** Determina se e con quale intensità il segnale elettrico viene trasmesso lungo l'assone.

**B. Esempio Decisionale "Festival Musicale"**

- **Variabili di Input:**
  - $x_1$: Meteo favorevole ($1 =$ Bel tempo, $0 =$ Pioggia).
  - $x_2$: Compagnia ($1 =$ Gli amici partecipano, $0 =$ Nessun amico).
  - $x_3$: Trasporto agevole ($1 =$ Mezzi comodi, $0 =$ Viaggio complesso).
- **Meccanismo Decisionale:** I pesi $w_i$ rappresentano le preferenze individuali. Se il peso $w_1$ (meteo) è dominante, un valore $x_1 = 0$ impedirà il superamento della soglia $b$, portando l'output $y = g(w^T x + b)$ a zero.

---

## MODULO III — PARTE 4: Forward Pass, Backpropagation e Funzioni di Costo

**1. Forward Propagation (Propagazione in Avanti)**

Considerando una rete a $N$ strati, la propagazione in avanti calcola sequenzialmente le attivazioni per ogni livello $l = 1, \dots, N$:

$$z^l = W^l a^{l-1} + b^l$$

$$a^l = g(z^l)$$

con le condizioni al contorno $a^0 = x$ (input della rete) e $a^N = \hat{y}$ (output finale).

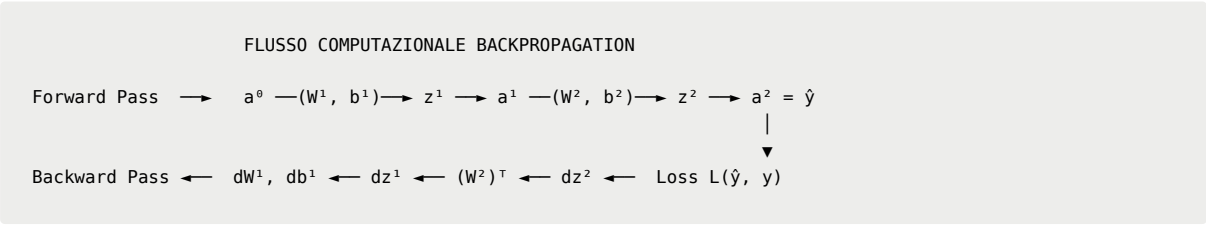
---

**2. Derivazione Matematica della Backpropagation (Retropropagazione dell'Errore)**

La funzione di costo globale $J(\theta)$ calcola la media della perdita $L(\hat{y}, y)$ sull'intero dataset o su un mini-batch:

$$J(\theta) = \frac{1}{m} \sum_{k=1}^{m} L(f(x_k; \theta), y_k)$$

L'obiettivo è calcolare le derivate parziali $\frac{\partial J}{\partial W^l}$ e $\frac{\partial J}{\partial b^l}$ per ogni strato $l$.




**Step 1: Errore nello Strato di Uscita $N$ ($dz^N$)**

Sfruttando la regola della catena multivariata (*Chain Rule*):

$$dz^N = \nabla_{z^N} J = \nabla_{a^N} J \cdot \frac{\partial a^N}{\partial z^N}$$

$$(\mathbf{Eq.1}) \quad dz^N = \nabla_{a^N} J \odot g'(z^N)$$

dove $\nabla_{a^N} J$ è il gradiente della perdita rispetto alle attivazioni finali e $g'(z^N)$ rappresenta la derivata elemento per elemento della funzione d'attivazione d'uscita.

**Step 2: Retropropagazione dell'Errore dallo Strato $l+1$ allo Strato $l$ ($da^l, dz^l$)**

Data la relazione $z^{l+1} = W^{l+1} a^l + b^{l+1}$, si deduce la variazione della perdita rispetto alle attivazioni intermedie $a^l$:

$$da^l = \nabla_{a^l} J = \frac{\partial J}{\partial z^{l+1}} \frac{\partial z^{l+1}}{\partial a^l} = (W^{l+1})^T dz^{l+1}$$

$$(\mathbf{Eq.2}) \quad dz^l = da^l \odot g'(z^l) = \left((W^{l+1})^T dz^{l+1}\right) \odot g'(z^l)$$

- **Intuizione:** L'errore del livello $l$ varia in modo direttamente proporzionale all'errore del livello successivo $l+1$ proiettato all'indietro mediante la trasposta della matrice dei pesi $(W^{l+1})^T$.

**Step 3: Gradienti rispetto ai Parametri $W^l$ e $b^l$**

Ottenuto il vettore d'errore $dz^l$, i gradienti per la matrice dei pesi e per il vettore dei bias si calcolano come:

$$(\mathbf{Eq.3}) \quad db^l = dz^l$$

$$(\mathbf{Eq.4}) \quad dW^l = dz^l (a^{l-1})^T$$

---

**3. Mini-Batch Stochastic Gradient Descent (SGD)**

L'aggiornamento dei parametri nell'ottimizzazione Mini-Batch SGD segue la seguente procedura iterativa:

1. Campionare un mini-batch casuale di $m$ campioni dal dataset.
2. Eseguire la Forward Propagation ed effettuare la Backpropagation su ciascun campione per accumulare i gradienti $dW^l$ e $db^l$.
3. Aggiornare i parametri per ogni strato $l$:

$$W^l = W^l - \frac{\alpha}{m} \sum_{k=1}^{m} dW_k^l$$

$$b^l = b^l - \frac{\alpha}{m} \sum_{k=1}^{m} db_k^l$$

1. Iterare su tutti i mini-batch fino a completare l'epoca e ripetere per il numero di epoche prefissato.

---

**4. Principali Funzioni di Costo (Loss Functions)**

**A. Mean Squared Error (MSE)**

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

- **Ambito d'Uso:** Compiti di regressione con output continuo.
- **Proprietà:** Penalizza in modo quadratico gli errori di grande entità rispetto alle piccole deviazioni.

**B. Binary Cross-Entropy (BCE) Loss**

$$\mathcal{L}_{\text{BCE}} = -\left[ y \log(\hat{y}) + (1-y) \log(1-\hat{y}) \right]$$

- **Ambito d'Uso:** Classificazione binaria con target $y \in \{0, 1\}$, abbinata a un'attivazione Sigmoide nello strato d'uscita.
- **Comportamento Logaritmico:**
  - Se $y = 1$, la perdita si riduce a $-\log(\hat{y})$. Man mano che $\hat{y} \to 1$, la perdita tende a $0$; se $\hat{y} \to 0$, la perdita diverge a $+\infty$.
  - Se $y = 0$, la perdita è $-\log(1 - \hat{y})$. Se $\hat{y} \to 1$, la perdita diverge esponenzialmente.

**C. Categorical Cross-Entropy (CE) Loss**

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

- **Ambito d'Uso:** Classificazione multi-classe su $C$ categorie disgiunte, abbinata a uno strato d'uscita Softmax.
- **Interpretazione:** Misura la divergenza di Kullback-Leibler tra la distribuzione di probabilità reale $y$ (vettore *one-hot*) e la distribuzione predetta dalla rete $\hat{y}$.